

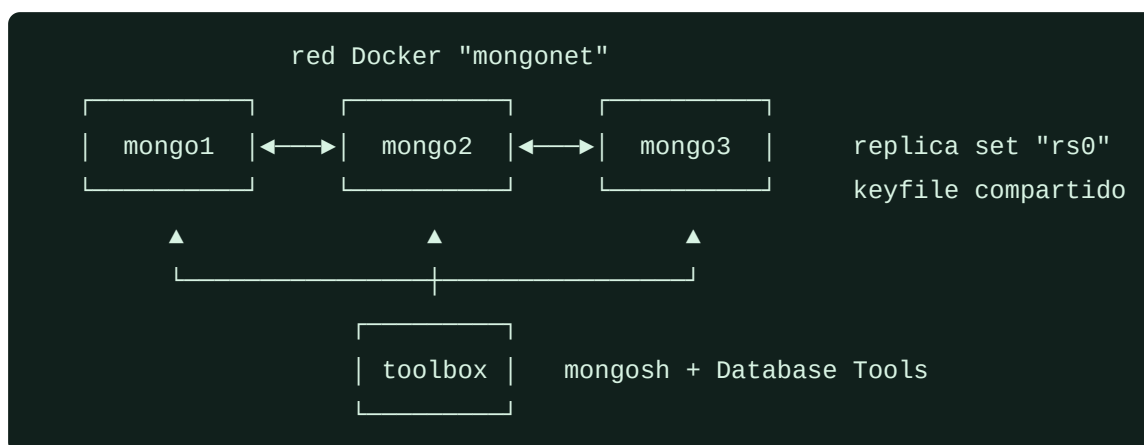
Laboratorio 05 · Replica set de tres nodos con autenticación interna

Duración: 40 minutos · Carpeta de trabajo: entorno/

Objetivos

Vas a desplegar un replica set de tres nodos con autenticación interna mediante *keyfile*, a iniciarlo a mano, a crear el usuario administrador aprovechando la localhost exception y a explorar el oplog y los heartbeats.

Arquitectura



Abre `entorno/compose.yml` y localiza: la imagen de los nodos, el parámetro `--replSet rs0`, el `--keyFile` y cómo se genera ese keyfile en el servicio `keygen`.

Paso 1 · Arrancar los tres nodos

```
cd entorno
docker compose up -d
docker compose ps
docker compose logs keygen
```

Mira el log de `mongo1`. Verás que el nodo arranca, pero está esperando una configuración de replica set:

```
docker compose logs mongo1 | grep -iE "replSet|config" | tail -5
```

Paso 2 · Iniciar el replica set

Nos conectamos **desde dentro de mongo1** porque, al tener `--keyFile`, la autorización ya está activa y todavía no existe ningún usuario: solo la localhost exception nos deja trabajar.

```
docker compose exec mongo1 mongosh
```

```
> rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "mongo1:27017" },
    { _id: 1, host: "mongo2:27017" },
    { _id: 2, host: "mongo3:27017" }
  ]
})
```

Pulsa Enter varias veces: el prompt cambiará de `rs0 [direct: other]` a `rs0 [direct: secondary]` y, tras la elección, a `rs0 [direct: primary]`. Comprueba quién es el primario:

```
> db.hello().primary
> db.hello().isWritablePrimary
```

Paso 3 · Crear el usuario administrador en el primario

Si `mongo1` es el primario, crea el usuario desde esta misma sesión. Si el primario es otro nodo, sal (`exit`) y conéctate a ese nodo con `docker compose exec mongo2 mongosh` (o `mongo3`).

```
> db.getSiblingDB("admin").createUser({ user: "admin", pwd:
"CursoMongo2026", roles: [ { role: "root", db: "admin" } ] })
> exit
```

El usuario se replica al resto de nodos como cualquier otro dato.

Paso 4 · Trabajar desde el toolbox

```
docker compose exec toolbox bash
echo "$RS"
mongosh "$RS"
```

La variable `RS` contiene la cadena de conexión al replica set completo. Carga los datos del curso:

```
mongosh "$RS" /scripts/datos.js
```

Paso 5 · Estado y configuración

```
> rs.status().members.map(m => ({ nodo: m.name, estado: m.stateStr,
salud: m.health, optime: m.optimeDate, ping: m.pingMs }))
> rs.conf()
> rs.conf().settings
```

1. ¿Cada cuánto se envían heartbeats (`heartbeatIntervalMillis`)?
2. ¿Tras cuánto tiempo sin noticias del primario se convoca una elección (`electionTimeoutMillis`)?
3. ¿Qué valor tiene `protocolVersion`?

Paso 6 · El oplog

```
> use local
> db.oplog.rs.stats({ scale: 1048576 }).maxSize
> rs.printReplicationInfo()
> rs.printSecondaryReplicationInfo()
```

Haz una modificación y búscala en el oplog:

```
> db.getSiblingDB("tienda").productos.updateMany({ categoria: "libros"
}, { $mul: { precio: 1.10 } })
> db.getSiblingDB("local").oplog.rs.find({ ns: "tienda.productos"
}).sort({ $natural: -1 }).limit(3)
```

1. El `updateMany` ha sido **una** operación para ti. ¿Cuántas entradas ha generado en el oplog?
2. ¿Qué contiene el campo `o` de cada entrada? ¿Aparece el `$mul`?

▼ Ver solución

Los heartbeats se envían cada 2 segundos (`heartbeatIntervalMillis: 2000`) y la elección se convoca tras 10 segundos sin contacto con el primario (`electionTimeoutMillis: 10000`). `protocolVersion` es 1: el protocolo de replicación basado en Raft que usa MongoDB desde la 3.2.

El `updateMany` genera **una entrada por documento modificado**. Además, el oplog no guarda el `$mul`, sino el resultado: en MongoDB 5.0+ verás entradas con `o: { $v: 2, diff: { u: { precio: ... } } }` y en `o2` el `_id` del documento. Esto hace que las

operaciones del oplog sean **idempotentes**: aplicarlas dos veces da el mismo resultado, algo imprescindible para que un secundario pueda reintentar la replicación tras un corte.

Paso 7 · Tamaño del oplog y ventana de replicación

Por defecto, el oplog ocupa el 5 % del espacio libre del disco, con un mínimo de 990 MB y un máximo de 50 GB. En producción debe cubrir, como mínimo, el tiempo máximo que un secundario puede estar caído o en mantenimiento sin necesitar una resincronización completa: la llamada **ventana del oplog** (*oplog window*), que `rs.printReplicationInfo()` muestra como `log length start to end`. El tamaño se puede cambiar en caliente, nodo a nodo:

```
> db.adminCommand({ replSetResizeOplog: 1, size: 2048 }) // en MB, se
aplica al nodo al que estás conectado
> rs.printReplicationInfo()
```

Para terminar

Deja el entorno en marcha para el laboratorio 06.

Si otro día necesitas reconstruir el replica set desde cero, hay un atajo que hace los pasos 2 y 3 automáticamente. En tu equipo:

```
docker compose exec mongo1 mongosh --quiet /scripts/bootstrap-rs.js
```

Y después, dentro del toolbox (`docker compose exec toolbox bash`):

```
mongosh "$RS" /scripts/datos.js
```